

ISE HN 2026 (endterm, 7 Aug 2026) — every exercise, solved and executable

The paper you sat on 7 Aug 2026: **1 h 40 min, 100 points**. Your graded result sheet is the best study guide there is, because it says exactly where marks went. Everything below is checked against **your** sheet.

Your result: 27 / 100, grade 5.0. Pass is 50, so you were 23 points short. The points you lost:

#	Exercise	Pts	Your result	Lost	Why (short)	Runnable here
1	Unit testing, CloudServer	20	4	16	the result of <code>allocateTask</code> was never stored (<code>NullPointerException ×4</code>), <code>%</code> instead of a cap, test arithmetic	<code>ise.testing.exam_cloudserver</code>
2	REST, Ticket Manager	25	0	25	"No commit was made" — nothing was pushed	<code>ise.rest.exam_ticket</code> (<code>gradlew runTicketServer</code> , port 8083)
3	Modeling, deployment diagram	20	4	16	only part of the diagram drawn, wrong stereotypes, lollipop on a communication path	\$3 below
4	Black-box	15	8	7	wrong classes, blank cell, ranges instead of single values in Part 3	<code>ise.blackbox.exam_graphics</code>
5	Quiz	20	11	9	see the table in §5	\$5 below

Two facts decide tomorrow:

- REST was 25 points and you got 0 because nothing was pushed.** The same thing cost 20–22 points in both earlier papers in your folder. Do REST early and `git push` after every endpoint.
- Unit testing: one missing line cost four tests.** You wrote in your complaint that it was "one omitted line repeated across methods". The answer was: *"Grading is per test method, and unfortunately, no partial credit is awarded if the test did not pass."* So the habit to build is `AllocationResult r = server.allocateTask(...)` in **every** test.

```
gradlew.bat test --tests "ise.testing.exam_cloudserver.*"
gradlew.bat test --tests "ise.rest.TicketResourceTest"
gradlew.bat test --tests "ise.blackbox.exam_graphics.*"
```

§1 — Unit testing: CloudServer (20 pts, 10 test cases)

Code: [CloudServer.java](#) · tests: [CloudServerTest.java](#).

The two methods — what went wrong in your submission (and one trap)

```
public AllocationResult allocateTask(ServerTier taskTier, double taskLoad) {
    boolean specialised = serverTier == taskTier;
    double actualRamCost = specialised ? taskLoad * 0.6 : taskLoad * 1.5;
    if (ram < actualRamCost) {
        return new AllocationResult(false, 0.0, 0.0);    // check BEFORE touching ram
    }
    ram -= actualRamCost;
    double processingSpeed = (optimizationLevel * 80.0) / (taskLoad + 10.0);
    if (specialised) processingSpeed += 40.0;
    return new AllocationResult(true, processingSpeed, actualRamCost);
}

public double freeRam(double amount) {
    if (amount <= 0.0) return ram;
    double effectiveRam = amount * (1.2 + (optimizationLevel * 0.05));
    ram = Math.min(ram + effectiveRam, 600.0 + (optimizationLevel * 40.0));    // CAP, not %
    return ram;
}
```

Bug in your submission	Why it is wrong
ram = ram % (600 + level*40.0)	% wraps around to a small number. "Never exceeds the cap" means Math.min. The grader's own test said: <i>"freeRam did not correctly cap the RAM at max capacity. expected: <1000.0> but was: <35.0>"</i> .
(trap, not your bug) optimizationLevel * 80 (int)	Fine here only because the divisor is a double. If both sides were int you would get integer division . Write 80.0.

The five tests — what was wrong in your submitted tests

#	Mistake in your submission	Fix
1	cloudServer.allocateTask(...) — return value thrown away , then allocationResult.isSuccess() on a never-assigned field → NullPointerException	AllocationResult result = server.allocateTask(...)
2	assertEquals(allocationResult.getActualRamCost(), (2*80)/(5+10)+40) — integer division (160/15 = 10), and cost compared against a <i>speed</i>	assertEquals((2 * 80.0) / (5.0 + 10.0) + 40.0, result.getProcessingSpeed(), 1e-9)
3	assertEquals(cloudServer.getRam(), 5*0.6) — 5*0.6 is the cost , not the RAM left (server started with 50)	assertEquals(50.0 - 5.0 * 0.6, server.getRam(), 1e-9)
4	No delta on doubles, arguments backwards (actual, expected)	assertEquals(expected, actual, delta)
5	testFreeRamNormal expected 10 * 1.3 = 13 — forgot the server already held 10	expected is 10 + 13 = 23
6	Your failed-allocation test used RAM 0 — passes even if the guard is wrong in many ways	Use RAM 10, task needs 15: proves "unchanged" is a real claim
7	Tests "not implemented yet" (fail("Test is not implemented yet"))	3 of your 5 lost marks. Write the easy test, push, move on.

Your six failing tests, decoded from the grader output:

Grader message	Meaning
Cannot invoke "AllocationResult.isSuccess()" because "this.allocationResult" is null (x4)	The return value of allocateTask(...) was thrown away and the field allocationResult was never assigned. One missing line, four dead tests.

Grader message	Meaning
expected: <490.0> but was: <680.0>	Your cap test: $78010 \% 680 = 490$ (the % bug) — and the arguments were the wrong way round, so the message even blames the right value.
expected: <23.0> but was: <13.0>	Your normal-release test compared the new RAM ($10 + 13 = 23$) against only the freed amount (13).

Your four passing tests were the "easy" ones; the rest scored nothing because **grading is per test method, no partial credit**.

Numbers worth having in your head (level = 2, as in the tests)

Scenario	Result
server 50 GB, HIGH_MEM, task HIGH_MEM load 5	cost $3.0 \cdot \text{RAM left } 47.0 \cdot \text{speed } 160/15 + 40 = \mathbf{50.666...}$
server 50 GB, STANDARD, task HIGH_MEM load 5	cost $7.5 \cdot \text{RAM left } 42.5 \cdot \text{speed } \mathbf{10.666...}$
server 10 GB, STANDARD, task HIGH_MEM load 10	needs 15 → fails , RAM stays 10, speed 0.0, cost 0.0
server 7.5 GB, task needing exactly 7.5	succeeds , ends at 0 (< not <=)
freeRam(10) from 10 GB	$10 + 10 \times 1.3 = \mathbf{23.0}$
freeRam(60000)	capped at $600 + 80 = \mathbf{680.0}$

The mutation drill proves these tests are strong: four broken CloudServers (modulo instead of cap, no +40 bonus, no same-tier discount, <= instead of <) each turn exactly one test red.

\$2 — REST: Ticket Manager (25 pts, ~2.8 pts per test — you got 0)

Code: [TicketService](#), [TicketResource](#) · tests: [TicketResourceTest](#) (11 tests, every row of the status table).

Your result was 0 / 25: "No commit was made" and "You didn't submit any solution for this exercise." Not one of the nine tests could run, so none of your knowledge counted. Do this in the first 5 minutes tomorrow: open the REST exercise, make the tiny change (even a comment), `git add .` && `git commit -m "start"` && `git push`, and then push after **every endpoint**.

For reference, these are the failures **another student's** graded sheet showed on this same exercise, so you can avoid them:

Grader message	Meaning	Cause and fix
Status expected: <400> but was: <200> (create)	POST with an id was accepted	<code>if (ticket.getId() != null) throw new ResponseStatusException(HttpStatus.BAD_REQUEST);</code>
Status expected: <200> but was: <500> (get one)	server crashed	usually <code>@PathVariable Long id</code> while the mapping says <code>{ticketId}</code> — names must match — or an <code>Optional.get()</code> on empty. Use <code>@PathVariable Long ticketId</code> .
Status expected: <400> but was: <200> (update)	id mismatch not checked	<code>if (!Objects.equals(ticketId, ticket.getId())) → 400</code>
Status expected: <204> but was: <500> (delete)	threw instead of returning	delete must be 204 always , even for unknown ids. <code>removeIf</code> , never <code>get()</code> .

Grader message	Meaning	Cause and fix
expected: not <null> (update service)	update branch returned null on success	return the stored ticket after copying fields; only return null when not found

The service, in the order the exam lists it

```
public Ticket saveTicket(Ticket ticket) {
    if (ticket.getId() == null) {
        ticket.setId(nextId); nextId++;           // assign, THEN increment
        tickets.add(ticket);
        return ticket;                             // not null!
    }
    Optional<Ticket> stored = findTicketById(ticket.getId());
    if (stored.isEmpty()) return null;             // update must FAIL, never create
    Ticket t = stored.get();
    t.setTitle(ticket.getTitle()); t.setDescription(ticket.getDescription());
    t.setPriority(ticket.getPriority()); t.setStatus(ticket.getStatus());
    return t;
}
```

getAllTickets() returns new ArrayList<>(tickets) (the TODO says a copy). findTicketById compares with .equals(), never == — Long objects above 127 are different objects.

The five endpoints

```
@RestController @RequestMapping("/tickets")
public class TicketResource {
    @PostMapping                                createTicket(@RequestBody Ticket t)                400
    if id != null, else 200
    @GetMapping("/{ticketId}")                  getTicket(@PathVariable Long ticketId)                404
    if absent
        @GetMapping                            getAllTickets()                                    200
        @PutMapping("/{ticketId}")              updateTicket(@PathVariable Long ticketId,
                                                    @RequestBody Ticket t)                400
    mismatch, 404 null from service
    @DeleteMapping("/{ticketId}")              deleteTicket(@PathVariable Long ticketId)              204
    always
}
```

POST returns **200 not 201** — the exam's table says 200. Follow the table, not REST convention.

§3 — Deployment diagram: the hospital (20 pts, you got 4)

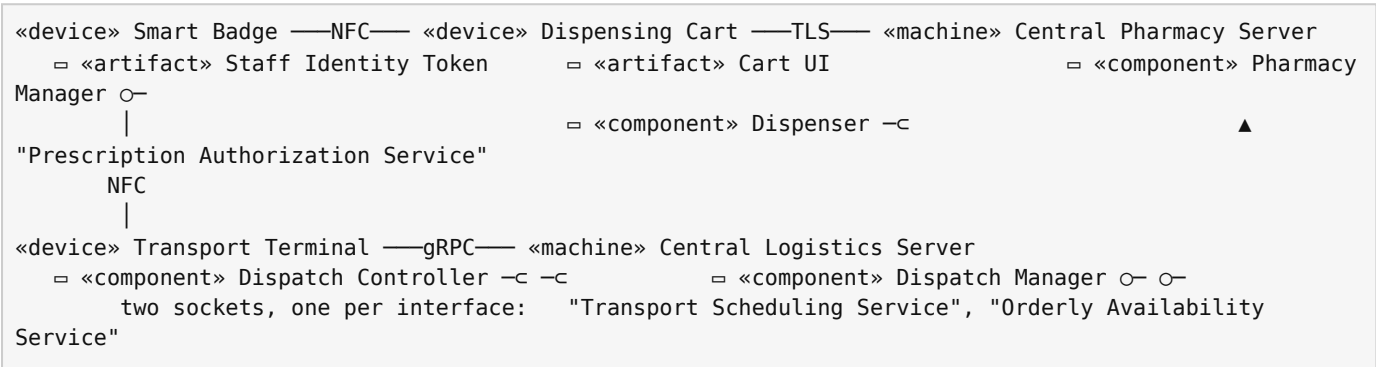
Your sheet gave +0.5 for each nested element placed correctly, +1 for each device drawn correctly (Dispensing Cart, Smart Badge, Transport Terminal), and these deductions:

What the grader said	Why	Fix
Staff Identity Token "should be modelled as an artifact, not a component"	The statement says "runs the <i>Staff Identity Token</i> artifact ". You drew a «component».	"runs ... artifact" → «artifact»; "deploys ... component" → «component»
NFC communication path drawn with a lollipop: "You should not have the lollipop. This is not an interface"	A communication path is a plain line between two nodes , labelled with the protocol. A lollipop is only for "requires an interface provided by a component".	NFC / TLS / gRPC = plain labelled lines. Interfaces only between components .

What the grader said	Why	Fix
Central Pharmacy Server "should be a machine node, not a device" (−0.5)	The statement says "Central Pharmacy Server is a machine ".	Copy the word: «machine»

The rest of the 20 points were for elements that did not appear in your diagram yet: the **Central Logistics Server**, the components (Dispenser, Dispatch Controller, Pharmacy Manager, Dispatch Manager), the **TLS** and **gRPC** paths, the second NFC path (Badge → Terminal) and the three named interfaces. A deployment diagram is graded element by element, so drawing **everything the statement lists** (and nothing else) is worth far more than drawing a few things beautifully. Budget 10 minutes, tick each sentence off.

The full solution



Statement says	You draw
"X is a device "	3-D box «device»
"X is a machine "	3-D box «machine» — not «node»
"runs the Y artifact "	plain box «artifact» nested inside the node
"deploys the Z component "	«component» box nested inside
"communicates securely with ... over TLS"	plain line between the two nodes , label TLS (no arrowhead)
"requires one interface provided by a component in ..."	socket ⊏ on the requiring component, lollipop ⊙ on the providing one, named after the service ("Prescription Authorization Service" — never "Interface")
"requires two interfaces"	two separate lollipop/socket pairs, two names

Rules the rubric rewards: nothing extra (extra nodes/paths are penalised), the socket points at the lollipop in the right direction (requirer → provider), and a model that is incomplete is not graded partially.

§4 — Black-box testing: Graphics Configurator (15 pts, you got 8)

Executable: [GraphicsConfiguratorBlackBoxTest](#) — every row below is a real assertion.

The submission, filled in (paste exactly this)

```
# Part 1: Equivalence Classes

### `deviceType`

| Class | Valid/Invalid | Inputs |
|---|---|---|
| DC1 | Valid | `Mobile` |
| DC2 | Valid | `Console` |
| DC3 | Invalid | any other value or empty input, e.g. `Handheld`, `PC`, `` |

### `vramMB`

| Class | Valid/Invalid | Inputs |
|---|---|---|
| VC1 | Invalid | `vramMB <= 2047` |
| VC2 | Valid | `deviceType = Mobile` with `2048 <= vramMB <= 4096` |
| VC3 | Valid | `deviceType = Mobile` with `4097 <= vramMB <= 16384` |
| VC4 | Valid | `deviceType = Console` with `2048 <= vramMB <= 8192` |
| VC5 | Valid | `deviceType = Console` with `8193 <= vramMB <= 16384` |
| VC6 | Invalid | `vramMB >= 16385` |

# Part 2: Representative Test Cases

| Test Case | deviceType | vramMB | Expected Result |
|---|---|---|---|
| TC1 | `Mobile` | `1000` | `unsupported` |
| TC2 | `Mobile` | `3000` | `performance` |
| TC3 | `Mobile` | `6000` | `quality` |
| TC4 | `Console` | `5000` | `performance` |
| TC5 | `Console` | `12000` | `quality` |
| TC6 | `Console` | `24000` | `unsupported` |
| TC7 | `Handheld` | `6000` | `unsupported` |

# Part 3: Boundary Values

#### Mobile Minimum Engine Baseline
*Tested with `deviceType = Mobile`:*

| Test Case | vramMB | Expected Result |
|---|---|---|
| TC10 | `2047` | `unsupported` |
| TC11 | `2048` | `performance` |
| TC12 | `2049` | `performance` |
```

Where your 7 points went — read this twice

Your answer	Grader's comment	Correct
VC2 = Invalid, vramMB > 8192	"The invalid class for exceeding the ceiling (vramMB >= 16385) is necessary"	VC6 is vramMB >= 16385, for both devices
VC5 = Invalid, Mobile with 2048 > vramMB or vramMB > 4096	"2048 > vramMB already exists in VC1. vramMB > 4096 up to 16384 is a valid class: Mobile quality"	VC3 / VC5 style classes are valid bands , never "everything outside the band"
VC6 left as <fill> with Console text	no marks	Console quality 8193 <= vramMB <= 16384. Every <fill> must be replaced.

Your answer	Grader's comment	Correct
TC1 = Mobile, 70000, unsupported	"redundant with TC6, which covers vramMB exceeding the ceiling"	TC1 is the low invalid class VC1: Mobile, 1000, unsupported
TC2 = Console, 3000, 'inclusive'	wrong expected result	performance — the expected result must be one of performance / quality / unsupported , never a comment. Your complaint was rejected.
TC5 Console 12000 left blank	no marks	quality — a blank cell is zero
TC10 = 2047>= vramMB, TC11 = 2048<= vramMB <= 16384, TC12 = vramMB >= 16385 with supported	"Boundary limits are not correctly specified; expected 2047, 2048, 2049"	Part 3 wants three single numbers around one limit: 2047 → unsupported, 2048 → performance, 2049 → performance. Not ranges, not three different limits.

What you **did** get right (so keep doing it): DC1, DC3, VC3-as-Mobile-performance, VC4-as-Console-performance, TC3, TC4, TC6, TC7.

The rule: read the heading of Part 3 ("Mobile Minimum Engine Baseline"), find that limit in the text (≤ 2047 fails $\Rightarrow$ first valid value 2048), then write limit−1, limit, limit+1. Don't wander to a different boundary.

Other rules proven by the test: one invalid input at a time (TC1, TC6 and TC7 each break only one parameter), Mobile and Console are case-sensitive, VC1/VC6 apply to both devices.

§5 — Quiz: all 18 questions, with your score (11 / 20)

#	Topic	Right answer	Your score
1	Software characteristics (select all)	True: "Software has no natural locality", "SE must create suitable forms of distance and isolation", "reuse can be lucrative when little customisation is needed". False: "memory provides protective distance", "reuse is always inexpensive"	0.2 / 1
2	Functional vs non-functional	Functional (what it does): grade quizzes, issue certificate, enrol in courses. Non-functional (how well): usable by visually impaired, stream without buffering on 5 Mbps, back up every 24 h	0.7 / 1
3	INVEST — "error messages should be <i>helpful</i> "	Testable ("helpful" cannot be verified)	0 / 1
4	git add README.md → state?	Staged (modified → <i>add</i> → staged → <i>commit</i> → committed)	0 / 1
5	Which line checks 0 °C = 32 °F	assertEquals(32.0, result, 0.001) and assertTrue(result == 32.0)	0.8 / 1
6	Same login setup repeated across acceptance tests	Common Includes (Parameterized Includes = same logic, different data)	0 / 1
7	Users can't figure out basic tasks	Usability	0 / 1
8	New backend on real traffic, users see nothing	Dark Launching (Canary = a small share of users see it; Blue-green = switch whole environments; Feature toggle = a flag)	0 / 1

#	Topic	Right answer	Your score
9	Cognitive complexity increases when...	linear flow is interrupted and nesting depth increases (declaring a method or long names add nothing)	0.8 / 1
10	Catch vulnerabilities / null pointers on every PR before build	Automated Static Analysis (ASA)	1 / 1
11	Effective review meeting	Focus on finding and logging defects; defer solutions	1 / 1
12	Process definitions (select all)	True: incremental = several usable parts; Waterfall = largely sequential phases; Kanban = board, cards move left to right, WIP limited . False: incremental = whole system once at the end; Kanban = fixed sprints + sprint review (that is Scrum); "Product Owner facilitates, removes impediments" (that is the Scrum Master)	1.7 / 2
13	Match scenario to model property (abstraction/reduction, pragmatics, descriptive, mapping, prescriptive)	The scenarios were drag-and-drop and not readable in the PDF, so here are the definitions to decide by: Mapping = the model represents an original; Abstraction/Reduction = it leaves out irrelevant details; Pragmatics = it is made for a purpose, an audience and a time; Descriptive = shows what exists; Prescriptive = prescribes what should be built	0.2 / 1
14	Match architecture principles	Information hiding · High cohesion · Modularization · Low coupling · Separation of concerns (all matched correctly)	2 / 2
15	UserService does new MySQLDatabase()	Violates Dependency Inversion (high-level depends on a concrete class); fix: a Database interface injected into UserService. Not SRP, not Liskov.	0.8 / 1
16	Core Domain	Represents the organisation's main competitive advantage	0 / 1
17	Maintenance adapts software to changes in...	Requirements and Technology	1 / 1
18	PDF exporter, new formats expected	An exporter interface with a separate implementation per format (Open-Closed)	1 / 1

Total: 11 / 20. **The seven questions you got 0 on were all one-line facts** (INVEST, git stages, Common Includes, Usability, Dark Launching, Core Domain, Software characteristics). They are in `CHEAT_SHEET.md` section 6 — learn that table and you gain about 6 points for free.

What to do differently tomorrow (the order that earns points)

1. **Clone, make a trivial change, commit, push** — before reading anything (step 12 of the BIE setup).
2. Quiz first, 15 minutes max.
3. **REST second** (25 pts, biggest block). Push after **every** endpoint.
4. Unit testing: write the **two easy tests first**; never leave `fail("not implemented")`. For every test: assign the result, assert it, delta on doubles, expected first.
5. Black-box: copy the heading of Part 3, find that exact limit.
6. Modeling last: copy words from the statement exactly (`«machine»` vs `«node»`), one line per "communicates".